

p1_01_sizeof.cpp · 内存结构图

按本机实测绘制 (Apple clang 21 / arm64 / macOS) : 1 格 = 1 字节, 数据成员、填充、vptr 分色。

数据成员 对齐填充 vptr (虚表指针) 占位字节

Empty 空类: 为什么不是 0 字节 sizeof = 1 字节



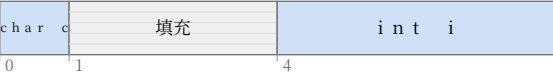
C++ 要求每个对象都有唯一地址, 空类也必须占 1 字节。

OneChar 只含一个 char sizeof = 1 字节



char 对齐 = 1, 所以大小就是 1, 没有任何填充。

CharInt char 在前, int 在后 sizeof = 8 字节



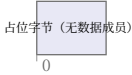
int 必须落在 4 的倍数上: char 占偏移 0, int 只能从偏移 4 开始, 中间 3 字节变成填充; 整体按 4 对齐。

IntChar int 在前, char 在后 sizeof = 8 字节



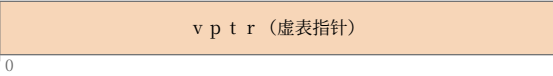
int 在偏移 0、char 在偏移 4; 整体必须按 4 对齐, 尾部又补 3 字节 —— 大小是 8, 不是 5。

HasFunction 只有非虚成员函数 sizeof = 1 字节



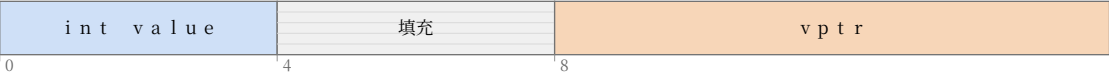
成员函数不占对象内存: 它编译成普通函数, 调用时额外把 this 传进去, 所以 sizeof 仍是 1。

HasVirtual 有一个虚函数 sizeof = 8 字节



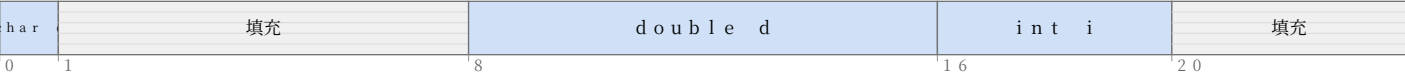
出现虚函数后对象里多出一个 vptr (64 位下 8 字节), 它指向该类唯一的虚表。

WithVirtual int + 两个虚函数 (例题 1-4) sizeof = 16 字节



数据成员先按对齐落位 (int 在 0, 补到偏移 8), 再放 vptr —— 16 字节。

LooseLayout char → double → int (例题 1-2) sizeof = 24 字节



double 要 8 对齐, char 后面跳 7 字节; int 之后整体再补到 8 的倍数 —— 24 字节。

TightLayout double → int → char (例题 1-2) sizeof = 16 字节



同样的三个成员, 只改了顺序: 16 字节, 比上面省 8 字节 —— 成员排序也是优化。

三条规则

- 成员按自身对齐要求落位: char=1、int=4、double=8、指针=8; 放不下就往后挪, 空洞变成填充。
- 结构体整体按最大成员对齐; 尾部常要补一段, 保证数组里每个元素都对齐 (IntChar 是 8 的原因)。
- 空类占 1 字节占位; 有虚函数多加 8 字节 vptr —— 这两条与「有几个成员」无关。